

Relatório - Módulo 3: Testes de Unidade

Atividades 1, 2 e 3 – Análise dos Testes do Projeto NoFluxo

Disciplina: FGA0314 – Testes de Software
Universidade de Brasília – Faculdade do Gama
Engenharia de Software

Maio de 2026

Sobre o projeto analisado

O projeto analisado é o **NoFluxo UnB**, uma aplicação web desenvolvida pela disciplina como parte do trabalho da turma. Ele é composto por:

- Um **backend Node.js/TypeScript** (`no_fluxo_backend`) que expõe controllers Express e usa Supabase como banco;
- Um **serviço Python** de IA (`ai_agent`) baseado em Flask, integrado ao RAGFlow;
- Scripts **Python** de scraping/coleta de dados do SIGAA;
- Frontends em **Flutter** e **Svelte**.

A análise se concentrou em duas áreas principais com testes *de unidade*: `no_fluxo_backend/tests-ts/` (Jest + TypeScript) e `tests-python/` (Pytest + unittest).

1 Atividade 1 – Anatomia dos Testes do Projeto

O objetivo desta atividade é mapear como o projeto organiza seus testes de unidade. Foram analisadas as quatro dimensões pedidas no enunciado: convenção de nomes, estrutura AAA, uso de mocks/stubs e dependências externas.

1.1 Estrutura geral dos testes

Área	Framework	Localização
Backend TS (Express)	Jest 29.7.0 + ts-jest	<code>no_fluxo_backend/tests-ts/*.test.ts</code>
Serviço Python (Flask)	Pytest + pytest-mock	<code>tests-python/test_*.py</code>
Scrapers Python	unittest + mock	<code>tests-python/test_extrair_*.py</code>

1.2 Convenção de nomes

No **backend TypeScript** segue-se a convenção do Jest com `describe` agrupando por controller e `it` descrevendo o comportamento em inglês na forma “*should + verbo + condição*”:

```
1 describe('CursosController', () => {
```

```

2  it('should return 200 with cursos data when successful', async () => {
    ... });
3  it('should return 500 when cursos query fails', async () => { ... });
4  it('should return 500 when credits_por_curso query fails', async ()
    => { ... });
5  });

```

Listing 1: cursos_controller.test.ts – linhas 22, 42, 74, 90

Nos testes Python a convenção é o prefixo `test_` seguido por uma descrição em português usando `snake_case`:

```

1 def test_analisar_materia_endpoint_sucesso(client, mocker):
2     """Testa o endpoint /assistente em um cen rio de sucesso..."""

```

Listing 2: tests-python/test_app.py – linha 55

Avaliação: a convenção do backend TS está alinhada às boas práticas (foco em comportamento). Já a Python está parcialmente alinhada – os nomes descrevem comportamento, mas em alguns testes herdados de `unittest` usa-se nomes muito genéricos como `test_remover_acentos` ou `test_coleta_dados`, que focam em *nome do método* e não no comportamento esperado, exatamente o problema descrito no slide 23 da aula.

1.3 Estrutura AAA (Arrange / Act / Assert)

Os testes do backend TS seguem AAA de forma **implícita**: o padrão está presente, mas *sem* comentários demarcando os blocos. Exemplo:

```

1  it('should return 200 with cursos data when successful', async () => {
2      // Arrange (sem coment rio no projeto)
3      const mockCursosData = [ { id_curso: 1, nome_curso: 'Administracao'
    }, ... ];
4      (SupabaseWrapper.get().from('cursos').select as jest.Mock)
5          .mockResolvedValueOnce({ data: mockCursosData, error: null })
6          .mockResolvedValueOnce({ data: mockCreditosData, error: null });
7
8      // Act
9      const handler = CursosController.routes["all-cursos"].value;
10     await handler(mockRequest as Request, mockResponse as Response);
11
12     // Assert
13     expect(statusSpy).toHaveBeenCalledWith(200);
14     expect(jsonSpy).toHaveBeenCalledWith([ ... ]);
15 });

```

Listing 3: cursos_controller.test.ts – linhas 42–72 (AAA implícito)

Nos testes Python o padrão AAA também aparece, mas é frequentemente **misturado**: o setup do cliente Flask é uma *fixture*, mocks podem aparecer espalhados, e em alguns casos há vários blocos de Act + Assert no mesmo teste.

1.4 Uso de mocks/stubs

Backend TS: usa exclusivamente *Jest mocks* (`jest.fn()`, `jest.mock()`, `jest.spyOn()`). A estratégia é mockar o módulo `SupabaseWrapper` no topo de cada arquivo de teste, evitando qualquer chamada real ao banco:

```

1 jest.mock('../src/supabase_wrapper', () => {
2   const mockSelect = jest.fn();
3   const mockFrom = jest.fn(() => ({ select: mockSelect }));
4   const mockGet = jest.fn(() => ({ from: mockFrom }));
5   return { SupabaseWrapper: { get: mockGet } };
6 });

```

Listing 4: cursos_controller.test.ts – linhas 6–20 (mock do Supabase)

Python: usa `unittest.mock` (`@patch`, `MagicMock`, `mock_open`) e o auxiliar `pytest-mock` (`mockers.patch`). Exemplo no `test_app.py`:

```

1 mock_ragflow_client = mockers.patch(
2   'no_fluxo_backend.ai_agent.app.RagflowClient'
3 ).return_value
4 mock_ragflow_client.start_session.return_value = "sessao_mock_123"
5 mock_ragflow_client.analyze_materia.return_value = {"code": 0, "data": "
   dados_mock"}

```

Listing 5: tests-python/test_app.py – linhas 60–63

1.5 Dependências externas

A tabela abaixo resume as dependências externas *detectadas* e como elas são tratadas nos testes:

Dependência	Onde aparece	Tratamento nos testes
Banco Supabase	Backend TS, todos os controllers	Mockado via <code>jest.mock</code>
Request/Response Express	Backend TS	Mock manual via <code>Partial<Request></code>
RAGFlow API (HTTP)	<code>ai_agent/app.py</code>	<code>mockers.patch</code>
SIGAA (HTTP scraping)	<code>coleta_dados/scraping/</code>	<code>@patch('requests.Session')</code>
Sistema de arquivos	Salvar/ler JSON e CSV	<code>mock_open</code> , <code>patch('builtins.open')</code>
Variáveis de ambiente	<code>config.py</code>	<code>monkeypatch</code> (pytest)
Relógio/data atual	Não há testes que dependam disso	N/A

Conclusão da Atividade 1: o projeto *tenta* seguir as boas práticas vistas em aula – nomeação por comportamento, AAA, mocks para isolar dependências externas. O backend TS é o mais alinhado; a parte Python tem mais inconsistências (nomes genéricos, AAA misturado) e alguns testes com setups muito longos.

2 Atividade 2 – Encontrar um “Teste Ruim”

2.1 Teste escolhido

O teste escolhido foi “should return 200 with database test results when successful” do bloco `describe('banco')`, no arquivo `no_fluxo_backend/tests-ts/testes_controller.test.ts`

(linhas 67–130). Trata-se de um teste *em produção* no projeto que acumula vários sintomas de “teste ruim”.

2.2 Código original

```
1 describe('banco', () => {
2   it('should return 200 with database test results when successful',
3     async () => {
4       const mockCursosData = [
5         { nome_curso: 'Administracao' },
6         { nome_curso: 'Engenharia' }
7       ];
8
9       const mockMateriasData = [
10        {
11          nome_curso: 'Administracao',
12          materias_por_curso: [
13            {
14              id_materia: 1,
15              nivel: 1,
16              materias: {
17                id_materia: 1,
18                nome_materia: 'Matematica',
19                codigo_materia: 'MAT001'
20              }
21            },
22            {
23              id_materia: 2,
24              nivel: 2,
25              materias: {
26                id_materia: 2,
27                nome_materia: 'Portugues',
28                codigo_materia: 'PORT001'
29              }
30            }
31          ]
32        }
33      ];
34
35      // Mock primeira chamada - cursos
36      const mockSelectChain1 = jest.fn().mockResolvedValueOnce({
37        data: mockCursosData,
38        error: null,
39      });
40
41      // Mock segunda chamada - materias_por_curso
42      const mockSelectChain2 = jest.fn().mockResolvedValueOnce({
43        data: mockMateriasData,
44        error: null,
45      });
46
47      const mockLimitChain = jest.fn().mockReturnValue({
48        select: mockSelectChain2,
49      });
50
51      mockFrom.mockReturnValueOnce({
52        select: mockSelectChain1,
```

```

53     select: jest.fn().mockReturnValue({
54         limit: mockLimitChain,
55     }),
56 });
57
58     const handler = TestesController.routes["banco"].value;
59     await handler(mockRequest as Request, mockResponse as Response);
60
61     expect(statusSpy).toHaveBeenCalledWith(200);
62     const responseData = jsonSpy.mock.calls[0][0];
63     expect(responseData.teste).toBe('conexao_banco');
64     expect(responseData.status).toBe('sucesso');
65 });
66 });

```

Listing 6: testes_controller.test.ts – linhas 67–130 (teste original “ruim”)

2.3 Problemas identificados

1. **Difícil de entender (mais de 60 linhas).** O teste tem ~65 linhas. A maior parte é setup de mock; o comportamento testado real (uma chamada e três asserts) está enterrado ao final.
2. **Múltiplos asserts verificando coisas diferentes.** `expect(statusSpy).toHaveBeenCalledWith(200)`, `expect(responseData.teste).toBe(...)` e `expect(responseData.status).toBe(...)` testam *três* aspectos distintos. Quando o teste falha, não fica claro *qual* comportamento foi violado – o nome do teste só promete “return 200”.
3. **Mistura Arrange/Act/Assert.** Não há separação clara entre os três blocos. O Arrange é gigante, sem comentários, e a inicialização `mockFrom.mockReturnValueOnce(...)` é encadeada com lógica de “primeira chamada/segunda chamada” que esconde a regra de negócio testada.
4. **Depende fortemente da implementação interna.** Os mocks modelam a forma exata como o controller chama o Supabase (`from -> select -> limit -> select`). Qualquer refactor interno que mantenha o mesmo comportamento externo *quebra* o teste – problema clássico discutido no slide 23 da aula (“foca na implementação”).
5. **Dados de teste excessivos e irrelevantes.** O teste cria duas matérias completas (Matemática, Português) com níveis e códigos – nada disso é verificado nos asserts. Isso aumenta a carga cognitiva sem agregar valor.
6. **Nome genérico, não comunica o cenário.** “should return 200 with database test results when successful” diz *o que* retorna, mas não *em que cenário* (cursos encontrados? base vazia? quantos itens?). Pelo padrão da aula (slide 24), o nome deveria descrever o comportamento de forma inequívoca.
7. **Não cobre valores limite.** Não há testes para: lista de cursos vazia, erro na segunda query (`materias_por_curso`), ou um único curso retornado. Apenas o caminho “feliz” está coberto.
8. **Setup duplicado entre testes do mesmo describe.** O mesmo `mockMateriasData` aparece praticamente igual em `describe('curso')` e `describe('casamento')`, indicando falta de *test data builders*.

3 Atividade 3 – Reescrita do Teste

A reescrita aplica AAA explícito, melhora a nomenclatura, separa o teste em casos focados (*um act, um comportamento*) e adiciona casos de valores limite (lista vazia, erro de banco), conforme pedido no enunciado.

3.1 Test data builder e helper de mock

Primeiro, extraímos os dados e o setup repetido para um helper. Isso reduz o ruído nos testes e remove duplicação.

```
1 // --- helpers no topo do arquivo de teste ---
2
3 const aCurso = (nome: string) => ({ nome_curso: nome });
4
5 const stubCursosQuery = (data: any, error: any = null) => {
6   const select = jest.fn().mockResolvedValueOnce({ data, error });
7   mockFrom.mockReturnValueOnce({ select });
8 };
9
10 const stubMateriasPorCursoQuery = (data: any, error: any = null) => {
11   const select = jest.fn().mockResolvedValueOnce({ data, error });
12   const limit = jest.fn().mockReturnValue({ select });
13   mockFrom.mockReturnValueOnce({
14     select: jest.fn().mockReturnValue({ limit }),
15   });
16 };
```

Listing 7: Helpers extraídos para reduzir setup

3.2 Testes reescritos

Cada teste agora foca em *um* comportamento, segue AAA explícito e cobre um cenário limite distinto.

```
1 describe('TestesController.banco (teste de conexao com o banco)', () =>
2   {
3     it('Banco_com_cursos_e_materias_retorna_status_sucesso', async () =>
4       {
5         // Arrange
6         stubCursosQuery([aCurso('Administracao'), aCurso('Engenharia')])
7         ;
8         stubMateriasPorCursoQuery([
9           {
10             nome_curso: 'Administracao',
11             materias_por_curso: [],
12           }
13         ]);
14
15         // Act
16         await TestesController.routes["banco"].value(
17           mockRequest as Request,
18           mockResponse as Response,
19         );
20
21         // Assert
22         expect(statusSpy).toHaveBeenCalledTimes(200);
23       }
24     );
25   }
26 );
```

```

19     expect(jsonSpy).toHaveBeenCalledWith(
20         expect.objectContaining({
21             teste: 'conexao_banco',
22             status: 'sucesso',
23         }),
24     );
25 });
26
27 it('
Banco_sem_cursos_cadastrados_retorna_status_sucesso_com_lista_vazia',
    async () => {
28     // Arrange -- valor limite: zero cursos
29     stubCursosQuery([]);
30     stubMateriasPorCursoQuery([]);
31
32     // Act
33     await TestesController.routes["banco"].value(
34         mockRequest as Request,
35         mockResponse as Response,
36     );
37
38     // Assert
39     expect(statusSpy).toHaveBeenCalledWith(200);
40     expect(jsonSpy).toHaveBeenCalledWith(
41         expect.objectContaining({
42             teste: 'conexao_banco',
43             status: 'sucesso',
44         }),
45     );
46 });
47
48 it('Banco_com_um_unico_curso_retorna_status_sucesso', async () => {
49     // Arrange -- valor limite: exatamente um curso
50     stubCursosQuery([aCurso('Engenharia')]);
51     stubMateriasPorCursoQuery([{ nome_curso: 'Engenharia',
materias_por_curso: [] }]);
52
53     // Act
54     await TestesController.routes["banco"].value(
55         mockRequest as Request,
56         mockResponse as Response,
57     );
58
59     // Assert
60     expect(statusSpy).toHaveBeenCalledWith(200);
61 });
62
63 it('
Banco_quando_query_de_cursos_falha_retorna_500_com_mensagem_de_erro',
    async () => {
64     // Arrange
65     const erro = { message: 'Database error' };
66     stubCursosQuery(null, erro);
67
68     // Act
69     await TestesController.routes["banco"].value(
70         mockRequest as Request,
71         mockResponse as Response,

```

```

72     );
73
74     // Assert
75     expect(statusSpy).toHaveBeenCalled(500);
76     expect(jsonSpy).toHaveBeenCalled({
77         teste: 'conexao_banco',
78         status: 'erro',
79         erro: erro.message,
80     });
81 });
82
83 it('
Banco_quando_query_de_materias_falha_retorna_500_com_mensagem_de_erro
', async () => {
84     // Arrange
85     stubCursosQuery([aCurso('Administracao')]);
86     stubMateriasPorCursoQuery(null, { message: 'materias error' });
87
88     // Act
89     await TestesController.routes["banco"].value(
90         mockRequest as Request,
91         mockResponse as Response,
92     );
93
94     // Assert
95     expect(statusSpy).toHaveBeenCalled(500);
96     expect(jsonSpy).toHaveBeenCalled(
97         expect.objectContaining({ status: 'erro' }),
98     );
99 });
100 });

```

Listing 8: Versão reescrita aplicando AAA, melhor nomenclatura e cobertura de valores limite

3.3 Melhorias realizadas

1. **AAA explícito.** Cada teste agora tem comentários `// Arrange`, `// Act` e `// Assert` demarcando os blocos – aumenta a legibilidade (slide 18 da aula).
2. **Nomes que descrevem comportamento.** Em vez de `should return 200 with database test results when successful`, usamos `Banco_com_cursos_e_materias_retorna_status_sucesso`. O nome descreve *o cenário* (com cursos e matérias) e *o resultado esperado* (status sucesso), seguindo a recomendação do slide 24 (“Use frases simples e expressivas”) e do slide 25 (separar palavras com *underscore*).
3. **Cobertura de valores limite.** O teste original cobria apenas o caminho feliz. A versão reescrita adiciona:
 - Lista de cursos *vazia* (zero elementos);
 - *Um único* curso (limite mínimo de uma coleção não vazia);
 - Falha na primeira query;
 - Falha na segunda query (`materias_por_curso`).

4. **Um Act por teste.** Cada teste tem exatamente uma chamada ao handler – prática recomendada no slide 13 (“Um bom teste normalmente possui apenas UM Act”).
5. **Asserts focados em comportamento, não em estrutura.** `expect.objectContaining({ status: 'sucesso' })` verifica apenas o que importa para aquele teste, em vez de exigir o JSON completo. Isso reduz a fragilidade quando campos auxiliares são adicionados/removidos da resposta.
6. **Helpers reduzem ruído de setup.** `stubCursosQuery` e `stubMateriasPorCursoQuery` encapsulam a forma como os mocks são encadeados, permitindo reuso entre testes e mantendo o foco do caso de teste no *cenário*, não no detalhe de implementação.
7. **Independência total.** Cada teste prepara seus próprios mocks (graças ao `afterEach` com `jest.clearAllMocks()`) e não depende da ordem de execução – atende ao requisito do slide 19 (“um teste deve poder executar sozinho, em qualquer ordem”).

Conclusão

A análise mostrou que o projeto NoFluxo já adota grande parte das boas práticas vistas no Módulo 3 – isolamento via mocks, AAA implícito e nomes razoáveis no backend TypeScript. Os pontos fracos detectados foram: (i) testes longos com setup excessivo, (ii) acoplamento dos mocks à implementação do Supabase, (iii) ausência de testes para valores limite e caminhos de erro nos testes do `TestesController`, e (iv) nomes que dependem do método e não do comportamento.

A reescrita demonstrou como aplicar diretamente as recomendações da aula (AAA, nomes expressivos, um Act por teste, cobertura de limites) *sem* reescrever o código de produção, ganhando legibilidade e cobertura ao mesmo tempo.